Building Artificial Intelligence for Computer Games

CSC484–001, Fall 2026
Course Project

Requirements

Where your homework behavior trees decide what a single character does, your project will decide what a *team* does; and where your homeworks run in environments you built by hand, your project will run in environments your own code generated.

You will implement the project in a game engine of your choosing. You may pick one of the engines used in Drexel University's game AI course, propose another engine you find, or continue in the C++ and `SFML` system you have been building in the homeworks if you want to stay with what we are doing in class. Whatever you choose, you will use the **same engine for all three deliverables**; this is one project, not three. If you choose an outside engine, the project does not depend on your homework code in any way, so you are encouraged to start early.

You are expected to contribute the following throughout the course of the semester:

1. Team and engine selection, required for approval but ungraded.
2. Deliverable 1: Strategic Decision Making, worth 30% of your project grade.
3. Deliverable 2: Procedural Content Generation, worth 30% of your project grade.
4. Deliverable 3: Putting it All Together, worth 40% of your project grade.

If you choose to do the project, you are to work in teams of 2-4. We recommend teams of 2, but larger teams will be considered if the proposal includes enough detail to explain why more people are needed on the team, to accomplish more technical goals.

As always, you are expected to abide by the University's Academic Integrity Policy (`http://policies.ncsu.edu/policy/pol-11-35-01`), which includes providing appropriate attribution for all external sources of information consulted while working on this project, including clear attribution, in your README and your code, of which code is the engine's, which is contributed by you , and which is contributed by others.

Team and Engine Selection

At this milestone you will submit your team proposed composition, what engine you will use, and a brief description of what you are thinking of doing. If you have more than 2 team members, you should also include a justification for having a larger team (e.g. a larger technical goal).

Good starting points are the engines from Drexel's game AI course, or your own homework system:

- **microRTS**, a minimal Java real-time strategy engine designed for AI research, with built-in scripted AIs and an annual competition: `https://github.com/Farama-Foundation/MicroRTS`

- **The Mario AI Framework**, a Java platformer with loadable levels and a collection of built-in agents: `https://github.com/amidos2006/Mario-AI-Framework`
- **S3**, a simple RTS, and the **A4Engine**, a rogue-like-style 2D engine, both distributed through Drexel's course pages: `https://www.cs.drexel.edu/~so367/teaching/2014/CS387-680/projectS3.html` and `https://www.cs.drexel.edu/~so367/teaching/2013/CS387-680/projectA4PCG.html`
- **C++ and SFML**, continuing the system from your homeworks; see `https://www.sfml-dev.org/learn.php`.

You may instead propose any engine of your choosing, provided it is open source and freely available to the graders.

Your engine must pass three tests, and your selection post must briefly argue that it does: 1) it exposes programmatic control of multiple AI-controlled agents; 2) it can load externally generated content (maps, levels, or an equivalent) rather than only built-in content; and 3) it supports, natively or with modest adaptation, a competitive scenario in the sense of Deliverable 1. Engines whose genre makes the canonical skirmish awkward (a platformer, for example) are not forbidden, but your selection post must propose an equivalent: two competing AI-controlled sides, a collectible or contested resource, and a win condition. If your engine cannot pass these tests, you want to discover that in September rather than November; that is what this milestone is for.

First come, first served on a given engine/scenario combination, so have a backup in mind. An unapproved engine earns no credit on any deliverable, and you may not change engines between deliverables.

What to submit: See Moodle for submission.

Deliverable 1: Strategic Decision Making

Approximate Due Date: 10/14/2026 by 11:59 PM see Moodle for official deadline

First, get your engine building and running from a clean checkout (if you are working in your homework system, make sure it is in working order), and stand up your competitive scenario: two sides of at least three AI-controlled units each; at least one resource that can be collected by a unit and delivered to that unit's home region (or your approved equivalent); and a win condition (perhaps first side to five deliveries, or most deliveries in two minutes). Use your engine's existing facilities wherever they exist; this part is about learning your engine, not rebuilding it. If you are working in your homework system, your Assignment 2–4 code plays that role.

Next, implement a **strategic module** for your side. Your strategy layer decides what your *team* does: it should observe the state of the scenario and assign each unit a role or goal (gather, intercept, defend, escort, or whatever fits your scenario), which unit-level control then carries out. Your strategist must be more than a fixed script; it should reassign roles as the scenario evolves. The big challenge in this part of the project is not any single algorithm, but the interface between levels: what information can you actually extract about the scenario state, and what commands can you actually issue? Make sure you devote a significant portion of your writeup to this point. It is crucial! *Hint:* to find algorithms for this part, try Google Scholar searches like "influence maps game AI," "real-time strategy game AI survey," "utility-based game AI," or "hierarchical task network real-time strategy" (make sure you cite anything you use!).

Then, implement or configure a **baseline opponent** for the other side. If your engine ships with simple built-in AIs, you may use one as your baseline (cite it clearly!); otherwise, script one. Run your strategist against your baseline. Who wins? How often? Devise a scheme to quantify performance over repeated matches and report on the results. If your strategist cannot beat your baseline, that is a finding. Analyze why!

Note: While there are open source implementations of strategic game AI (for example, existing microRTS tournament bots), they are not allowed for this part of the project. You must implement your own strategy layer. You may use your engine's built-in movement, pathfinding, and unit-control facilities; when the line between the two is unclear, ask.

Finally, write a 2–3 page single-spaced paper summarizing your findings. That is at least **two full pages**. Present a diagram of your strategy layer and its interface to the engine (a photo of a hand-drawing in a notebook is sufficient), and present data to support your performance claims. Screenshots of matches in progress are a must, and belong in an appendix, which does not count toward the page requirement.

Deliverable 1 is worth 30% of your project grade: the scenario (20%), the strategy layer (35%), the opponent and evaluation (20%), and the writeup (25%) of the deliverable grade.

What to submit: A .zip archive containing all of *your* source files, a README, and your writeup in .pdf format, uploaded to Moodle before the Moodle submission deadline.

Deliverable 2: Procedural Content Generation

Approximate Due Date: 11/12/2026 by 11:59 PM see Moodle for official deadline

Implement a **procedural generator** that produces content your engine can load and play: maps for an RTS, dungeons for a rogue-like, levels for a platformer, indoor environments for your homework system; produce whatever your engine consumes. Generated content should be comparable in complexity to the content your Deliverable 1 scenario ran in. Your generator must be parameterized (size, region count, obstacle or terrain density, resource placement, *etc.*). Running it with different parameters should produce recognizably different families of content, and running it with the same parameters should still produce variety. *Hint:* the literature is full of generation algorithms looking for a home. Try Google Scholar searches like "procedural content generation survey," "search-based procedural content generation," "cellular automata cave generation," "binary space partitioning dungeon generation," or "wave function collapse algorithm" (make sure you cite where you found whatever you use!).

Generated content is only useful if the engine and its agents can actually use it. Define **validity** for your content (every region reachable, required resources present, level completable) and check it automatically; the engine's own pathfinding, its agents, or a checker you write can serve as your validity oracle. Report what fraction of raw generator outputs are valid, and either repair or reject invalid ones. Then demonstrate the engine loading and playing your generated content with its AI running. Screenshots of agents operating in generated content are a must.

A generator is not a single artifact but a distribution of artifacts. Generate at least 100 pieces of content and characterize what your generator tends to produce. Choose at least three measurable properties (region count, path length between key locations, resource density, dead-end count; be creative), plot their distributions, and discuss. What can your generator produce? What can it *not* produce? How do your parameters move these distributions? *Hint:* a Google Scholar search for "expressive range analysis" will show you how others have done this kind of analysis.

Note: While there are open source procedural generation libraries (and your engine may even ship with a generator of its own), they are not allowed for this part of the project. You must implement your own generator. The engine's built-in generator, if it has one, makes a fine baseline for comparison.

Finally, write a 2–3 page single-spaced paper summarizing your findings, presenting your generation algorithm, your validity results, and your distribution analysis with supporting plots. That is at least **two full pages**. Screenshots of representative content (including an amusing failure or two) belong in the appendix.

Deliverable 2 is worth 30% of your project grade: the generator (35%), validity and integration (20%), the analysis (20%), and the writeup (25%) of the deliverable grade.

What to submit: A .zip archive containing all of *your* source files, a README, and your writeup in .pdf format, uploaded to Moodle before the submission deadline specified on Moodle.

Deliverable 3: Putting it All Together

Approximate Due Date: 12/04/2026 by 11:59 PM see Moodle for official deadline

Combine Deliverables 1 and 2: run your competitive scenario inside your generated content. Your strategist, your baseline opponent, and the engine's unit-level AI should all operate together in content none of them has seen before. Scenario elements (resources, starting positions, home regions) must be placed sensibly by the generator, and your validity oracle should guarantee it.

Design and run an **experiment**: your strategist versus your baseline across a *set* of generated content, at least 20 pieces. Does the content affect the outcome? Are there content properties (from your Deliverable 2 measures) that predict which side wins or how long matches last? If your engine has a community of existing AIs (microRTS, for example, has tournament bots), you may add one as a second opponent for comparison, clearly cited, and only as an opponent. Present data to support your analysis. *Hint:* for ideas on evaluating AI across generated content, try Google Scholar searches like "automated playtesting," "game balance evaluation," or "map balance real-time strategy." This is the payoff of the whole project, an AI technique evaluated across a distribution of environments rather than a single hand-built level, so give it the care it deserves.

Finally, write a 4–5 page single-spaced final report that presents the entire project: your engine and its architecture, your strategy layer, your generator, your integration, and your experimental results. That is at least **four full pages**. It is **strongly** suggested that you do not limit yourself to only answering those questions posed in this handout. Think creatively about what you have done. What other parameters can you tweak and what effect do they have on the results? The most successful reports will contain evidence that you have thought deeply about these techniques and what they can produce and have gone beyond what is written in this project description. As an appendix to your report, please include all relevant screenshots to support your analysis. The appendix does not count toward your 4–5 page requirement.

Deliverable 3 is worth 40% of your project grade: integration (25%), the experimental evaluation (30%), and the final report (45%) of the deliverable grade.

What to submit: A .zip archive containing all of *your* source files, a README, and your final report in .pdf format, uploaded to Moodl. If you are working in C++ and SFML, include one or more Makefiles and ensure your code compiles and runs in the standard grading environment; this applies to all three deliverables. If you are working in any other engine, your README must state the exact engine version you used and how to apply your code to it, and each deliverable

must include a short (2–4 minute) demo video showing your system in action, since the graders cannot be expected to run every engine. **Note: per the course syllabus, the late policy applies to Deliverables 1 and 2; Deliverable 3 may not be submitted late.**